

现代密码学课程设计说明书

- 学号: _____
 - 姓名: _____
 - 课程: 现代密码学 (同济大学 杨礼珍)
 - 题目: 基于 NTL 的加密、签名、证书方案、简易 PKI 与安全邮件系统设计
-

1. 课程设计任务概述

本课程设计基于开源大数据库 NTL (<http://www.shoup.net/ntl>), 使用 C++ 实现以下五个任务:

1. **RSA 加密和签名**: 密钥生成 (p, q 均为 1024 比特素数)、加密、解密、密钥格式转换、签名与签名验证;
2. **Elgamal 加密和签名**: 密钥生成、加密、解密、签名与签名验证;
3. **简单证书方案**: 证书 $\text{Cert}(\text{Alice}) = (\text{ID}(\text{Alice}) || \text{ver_Alice} || s || \text{ID}(\text{TA}) || \text{flag1} || \text{flag2})$, 其中 $s = \text{sig_TA}$ 的私钥 ($\text{ID}(\text{Alice}) || \text{ver_Alice}$), flag1 标识签名算法 (0=RSA, 1=Elgamal), flag2 标识公钥用途 (0= 加密, 1= 签名), 证书采用 txt 格式;
4. **简易 PKI 系统**: 教材 12.3.1 节严格层次 PKI 系统的简易版本——根 CA (自签名) 下挂两个下级 CA (CA1、CA2), 用户证书由下级 CA 签发; 实现可公开查询的证书库 (存储 CA 发来的证书, 按证书所有者 ID 返回证书路径 <CA_root 证书, CA1 证书, Alice 证书 >), 并给出一个使用例子 (Alice 发送消息与签名, Bob 查询并验证证书路径后用证书中的公钥验证签名);
5. **简易安全邮件系统**: 无需网络传输。发送端查询接收端加密公钥证书链并验证、对邮件 Hash 签名、用接收端加密公钥加密 $m || s$ 并发送; 接收端用加密私钥解密、查询发送端签名公钥证书链并验证、用发送端签名公钥验证签名。

不同对象 (TA、CA、证书库、Alice、Bob 等) 完成的任务均用**独立的模块** (独立的类与源文件) 完成, 并非用几个独立的程序完成。对以上方案和协议均有独立的测试函数。

2. 程序设计说明

2.1 总体结构与模块划分

项目使用 CMake 构建, C++17 标准, 依赖 NTL / GMP (位于 `third_party/local`)。源文件结构如下:

```
src/
|-- crypto/                                # 加密与签名引擎 (已有)
|   |-- crypto_engine.h/.cpp              # 抽象接口: PublicKey/PrivateKey/Keypair/EncryptionEngine/SignatureEngine
|   |-- rsa_engine.h/.cpp                 # RSA 密钥生成、加密解密、签名验证、密钥格式转换
|   |-- elgamal_engine.h/.cpp             # Elgamal 密钥生成、加密解密、签名验证
|-- hash/
|   |-- sha_256.h/.cpp                    # SHA-256 (FIPS 180-4), 用于签名时对消息取 Hash
|-- certificate/                           # 证书方案与 PKI 基础设施模块
|   |-- certificate.h/.cpp                 # 证书对象、txt 序列化、公钥文本转换、证书/证书路径验证
|   |-- trusted_authority.h/.cpp           # TA 模块: 为 Alice 颁发简单证书
|   |-- certificate_authority.h/.cpp       # CA 模块: 根 CA 自签名、为下级 CA 和用户签发证书
|   |-- certificate_library.h/.cpp         # 证书库模块: 只存储验证通过的 CA 证书, 按 ID 返回证书路径
|-- pki/
|   |-- pki_user.h/.cpp                   # 用户模块 (Alice/Bob/Eve): 加密/签名密钥对、向 CA 申请证书
|-- mail/
|   |-- secure_mail.h/.cpp                # 安全邮件模块: 发送端/接收端完整流程 (m||s 编码、加解密、验证)
|-- demo/
|   |-- umbra_demo.cpp                    # 演示程序 (可执行程序)
```

模块依赖关系：umbra_mail $\rightarrow$ umbra_certificate $\rightarrow$ umbra_crypto $\rightarrow$ umbra_hash。各对象（TA、CA、证书库、用户、邮件系统）的任务分别由独立模块完成；证书模块不再定义自己的 算法枚举或引擎工厂，证书中的签名算法（flag1）直接复用 crypto::CryptoType（0=RSA, 1=Elgamal），抽象类只保留在底层的 crypto 引擎接口中，上层证书/TA/CA/证书库/用户/ 邮件各模块均为普通的具体类与函数。

2.2 RSA 加密与签名设计（任务 1）

- **密钥生成**：随机生成两个 1024 比特素数 p 、 q （NTL::GenPrime_ZZ 内部执行足够多次 Miller-Rabin 素性检测，保证通过检测的数是素数的概率足够高），取 $e = 65537$ ，计算 $n = p \cdot q$ ， $d = e^{-1} \bmod \phi(n)$ 。公钥 (n, e) ，私钥 (p, q, d) 。
- **加密/解密**：将明文按 NumBytes(n) - 1 字节分组，每块转为大整数后做 $c = m^e \bmod n$ ，输出格式为每行块长-c；解密 $m = c^d \bmod n$ 。对解密/验证时来自外部（密文、签名文本）的大整数先约化到模数范围内，避免 NTL 的 PowerMod/InvMod 对越界输入直接终止程序，保证篡改输入时只返回 失败结果而非崩溃。
- **格式转换**：PublicKey(const std::string&) / operator std::string() 实现 公钥 n\ne、私钥 p\nq\nd 与文本的互转。
- **签名/验证**：对消息取 SHA-256 Hash 值 $H(m)$ ， $s = H(m)^d \bmod n$ ；验证时恢复 $H(m)' = s^e \bmod n$ ，与本地计算的 $H(m)$ 比较。

2.3 Elgamal 加密与签名设计（任务 1）

- **密钥生成**：生成安全素数 $p = 2q + 1$ （ q 为素数），取二次剩余子群的生成元 α （阶为 q ），私钥 a 随机取自 $[1, q-1]$ ，公钥 $(p, \alpha, \beta = \alpha^a \bmod p)$ 。公钥 (p, α, β) ，私钥 (p, q, a) 。
- **加密/解密**：每块 m ，取临时密钥 k ，计算 $c1 = \alpha^k \bmod p$ 、 $c2 = m \cdot \beta^k \bmod p$ ，输出 块长-c1-c2；解密 $m = c2 \cdot (c1^a)^{-1} \bmod p$ 。
- **签名/验证**：对 $H(m)$ （约化到模 q ），取 $\gcd(k, q) = 1$ 的随机 k ，计算 $r = \alpha^k \bmod p$ 、 $s = k^{-1}(H(m) - a \cdot r) \bmod q$ ；验证 $\alpha^{H(m)} = r^r \cdot \beta^s \pmod p$ 。验证时对签名文本中的 r 先约化到模 p 范围内，因此用错误公钥验证签名只会返回“不通过”，而不会触发 NTL 内部错误。

2.4 简单证书方案设计（任务 1）

证书格式 (txt)：

```
BEGIN UMBRA CERTIFICATE
ISSUER:TA
SUBJECT:Alice
ALGORITHM:RSA
PURPOSE:Encryption
PUBLIC_KEY:RSA <n> <e>
SIGNATURE:
<s>
END UMBRA CERTIFICATE
```

- 被签名的内容 signed_payload() = ID(Alice) || ver_Alice（即 SUBJECT 行与 PUBLIC_KEY 文本的拼接），签名 $s = \text{sig_TA}$ 的私钥 (signed_payload())；
- ALGORITHM (flag1): 0=RSA, 1=Elgamal（直接使用 crypto::CryptoType 枚举，值与之对应，不再单独定义算法枚举）；
- PURPOSE (flag2): 0= 加密, 1= 签名 (KeyPurpose 枚举值与之对应)；
- 公钥文本首行为类型标记 (RSA 或 ELGAMAL)，使证书中可存放任何一种公钥；
- 验证: ver_TA 的公钥 (signed_payload(), s) == true 则证书可信 (verify_certificate_signature)；
- TA 模块 (TrustedAuthority) 在构造时生成自己的签名密钥对，grant_certificate 负责签发证书，支持 RSA 与 Elgamal 两种签名算法，register_user 登记用户信息。

2.5 简易 PKI 系统设计（任务 4）

层次结构（教材 12.3.1 节）：

CA_root（证书由自己签发，自签名）

```
|-- CA1 -- Alice, Eve
|-- CA2 -- Bob
```

- **CA 模块** (CertificateAuthority)：每个 CA 持有自己的签名密钥对；self_sign() 为根 CA 生成自签名证书；issue_certificate() 为下级 CA 或用户签发证书（签发者用自己的私钥签名 ID(subject) || ver_subject）。
- **证书库模块** (CertificateLibrary)：只存储由 CA 颁发且签名验证通过的证书——自签名证书必须为受信根 CA 的证书，其余证书必须由其签发者（证书已在库中）的公钥验证通过；查询接口按证书所有者 ID 返回证书路径（证书链）：从叶证书沿签发者逐级向上直至自签名根证书，即 <CA_root 证书, CA1 证书, Alice 证书 >；另提供按 (ID, 用途) 的查询供邮件系统使用。
- **证书路径验证** (verify_certificate_path)：先用受信根公钥验证根证书（自签名），再逐级用上一级证书中的公钥验证下一级证书的签名。
- **使用例子**：Alice 用其签名私钥对消息签名；Bob 在证书库中查询 Alice 的证书路径，验证路径正确后，取出 Alice 证书中的签名公钥验证 Alice 的签名。

2.6 简易安全邮件系统设计（任务 5）

m || s 的编码采用“长度前缀”方式：signed_mail = to_string(len(m)) + '\n' + m + s, 其中 s = sig_发送端签名私钥 (SHA-256(m))。

- **发送端流程** (mail::send_mail)：
 1. 在证书库中查询接收端的**加密公钥**证书链，并验证证书链是否正确；不正确则抛出异常，由交互层决定重新申请证书链或退出；
 2. 用发送端签名私钥对邮件 m 的 Hash 值签名得到 s；
 3. 从接收端证书链的叶证书中取出接收端的加密公钥；
 4. 用接收端加密公钥加密 m||s 得到密文 c；
 5. 返回（发送）c。
- **接收端流程** (mail::receive_mail)：
 1. 用接收端加密私钥解密 c，得到 m||s；
 2. 在证书库中查询发送端的**签名公钥**证书链并验证；
 3. 从叶证书中取出发送端的签名公钥；
 4. 用发送端签名公钥验证签名，返回恢复出的邮件内容 m。

3. 程序使用说明

3.1 编译

项目依赖 NTL 与 GMP（已预编译于 third_party/local）。使用 CMake 构建：

```
# Debug 构建（含测试）
cmake --preset debug
cmake --build build/debug
ctest --test-dir build/debug --output-on-failure # 运行全部测试

# Release 构建
cmake --preset release
cmake --build build/release
```

3.2 测试程序

共 9 个测试目标（可执行程序），每个模块都有独立的测试：

测试程序	对应模块	测试内容
umbra_crypto_rsa_engine_test	RSA 引擎	RSA 文件加解密往返、签名/验证正反例
umbra_crypto_elgamal_engine_test	Elgamal 引擎	Elgamal 文件加解密往返、签名/验证正反例
umbra_hash_sha_256_test	SHA-256	SHA-256 标准向量与流式一致性
umbra_certificate_test	证书 (Certificate)	证书 txt 序列化、flag 数值、公钥文本转换、证书签名验证
umbra_trusted_authority_test	TA	TA(RSA/Elgamal) 签发与验证、flag2 用途、篡改/错误密钥拒绝
umbra_certificate_authority_test	CA	根 CA 自签名、为用户/下级 CA 签发证书
umbra_certificate_library_test	证书库	证书路径查询与验证、伪造/篡改拒绝、Alice→Bob 使用例子
umbra_pki_user_test	用户 (Alice/Bob)	向 CA 申请双证书、签名/验证
umbra_secure_mail_test	安全邮件	邮件发送 → 接收往返、篡改密文/错误接收者密钥/未知发送者/不可信链拒绝

3.3 演示程序 umbra_demo

用法：

`./build/debug/umbra_demo` [输入文件] [输出目录]

- **输入文件**（可选）：RSA/Elgamal 加密与安全邮件使用的明文文件（大量输入采用文件方式，符合任务书要求）；缺省时使用内置示例邮件。
- **输出目录**（可选）：存放密钥、证书、证书链、密文、恢复邮件等产物，缺省为 `./demo_output`。

运行后依次演示：RSA 加密签名 → Elgamal 加密签名 → 简单证书方案（TA 签发）→ 简易 PKI 系统（含使用例子）→ 简易安全邮件系统。主要产物及格式如下：

产物	格式
rsa/rsa_public_key.txt	n 换行 e
rsa/rsa_private_key.txt	p 换行 q 换行 d
rsa/rsa_ciphertext.txt	每行 块长-c
rsa/rsa_signature.txt	s
elgamal/elgamal_public_key.txt	p 换行 alpha 换行 beta
elgamal/elgamal_ciphertext.txt	每行 块长-c1-c2
certificates/*.cert	见 2.4 节证书 txt 格式
pki/root.cert、pki/ca1.cert、pki/alice/*.cert 等	各 CA 与用户证书
pki/alice_chain.txt	Alice 的证书路径（证书链）
mail/mail_ciphertext.txt	密文 c（每行 块长-c1-c2 或 块长-c）
mail/mail_recovered.txt	Bob 解密并验证后的邮件内容

3.4 运行示例（截图位置）

以下位置请粘贴实际运行截图：

1. ctest --test-dir build/debug --output-on-failure 全部通过的截图;
2. umbra_demo 运行 [任务 1] RSA 加密和签名部分截图;
3. umbra_demo 运行 [任务 1] Elgamal 加密和签名部分截图;
4. umbra_demo 运行 [任务 1] 简单证书方案部分截图 (含证书文件内容);
5. umbra_demo 运行 [任务 4] 简易 PKI 系统部分截图 (含证书路径与验证结果);
6. umbra_demo 运行 [任务 5] 简易安全邮件系统部分截图 (含发送端/接收端输出)。

4. 测试与运行结果

ctest --test-dir build/debug --output-on-failure 输出:

```
Test project /Users/tonhie/Projects/Umbra-Core/build/debug
Start 1: crypto.rsa_engine
1/9 Test #1: crypto.rsa_engine ..... Passed    0.01 sec
Start 2: hash.sha_256
2/9 Test #2: hash.sha_256 ..... Passed    0.00 sec
Start 3: crypto.elgamal_engine
3/9 Test #3: crypto.elgamal_engine ..... Passed    0.18 sec
Start 4: certificate.certificate
4/9 Test #4: certificate.certificate ..... Passed    0.02 sec
Start 5: certificate.trusted_authority
5/9 Test #5: certificate.trusted_authority ..... Passed    0.14 sec
Start 6: certificate.certificate_authority
6/9 Test #6: certificate.certificate_authority ... Passed    0.04 sec
Start 7: certificate.certificate_library
7/9 Test #7: certificate.certificate_library ..... Passed    0.43 sec
Start 8: pki.pki_user
8/9 Test #8: pki.pki_user ..... Passed    0.05 sec
Start 9: mail.secure_mail
9/9 Test #9: mail.secure_mail ..... Passed    0.79 sec
```

100% tests passed, 0 tests failed out of 9

umbra_certificate_test (证书测试, 节选):

```
=== Certificate Tests ===
PASS: certificate txt round trip (RSA)
PASS: certificate txt round trip (Elgamal)
PASS: signed payload is ID(Alice) || ver_Alice
PASS: flag1 / flag2 numeric values
PASS: RSA public key text round trip
PASS: certificate signature verifies with the issuer key
PASS: tampered certificate rejected
...
All certificate tests PASSED
```

umbra_trusted_authority_test (TA 测试, 节选):

```
=== Trusted Authority Tests ===
PASS: TA(RSA) certificate fields
PASS: TA(RSA) certificate signature verifies
PASS: tampered TA(RSA) certificate rejected
PASS: wrong TA(RSA) key rejected
PASS: TA(Elgamal) certificate signature verifies
PASS: flag2 (key purpose) is issued as requested
```

```

...
All trusted authority tests PASSED

umbra_certificate_library_test (证书库测试, 节选):

=== Certificate Library Tests ===
PASS: Alice chain order is <CA_root, CA1, Alice>
PASS: Alice chain verifies against the root public key
PASS: Bob chain order is <CA_root, CA2, Bob>
PASS: certificate from unknown CA rejected
PASS: self-signed certificate of a non-root CA rejected
PASS: certificate forged with a wrong CA key rejected
PASS: tampered leaf certificate fails chain verification
PASS: tampered intermediate CA certificate fails chain verification
PASS: Bob verifies Alice's certificate path
PASS: Bob verifies Alice's signature with the certified public key
PASS: Bob rejects a tampered message
...
All certificate library tests PASSED

umbra_secure_mail_test (安全邮件测试, 节选):

=== Secure Mail Tests ===
PASS: sender produces a ciphertext
PASS: receiver recovers the original mail
PASS: m || s splits back into m
PASS: tampered ciphertext fails on the receiver side
PASS: wrong receiver private key cannot recover the mail
PASS: sender without a certificate is rejected
PASS: chain not rooted at the trusted root is rejected
...
All secure mail tests PASSED

umbra_demo 运行结果 (节选):

[任务4] 简易 PKI 系统 (严格层次结构)
...
Alice 的证书路径 (证书链):
- CA_root (签发者: CA_root, flag1=0 [0=RSA,1=Elgamal], flag2=1 [0=加密,1=签名])
- CA1 (签发者: CA_root, flag1=0 [0=RSA,1=Elgamal], flag2=1 [0=加密,1=签名])
- Alice (签发者: CA1, flag1=1 [0=RSA,1=Elgamal], flag2=1 [0=加密,1=签名])
证书路径验证: 通过
...
[任务5] 简易安全邮件系统 (无网络传输)
[发送端 Alice] 步骤1: 在证书库中查询 Bob 的加密公钥证书链并验证...
    证书链验证: 通过
[接收端 Bob] ...
Bob 解密并验证后的邮件内容: ...
邮件完整性与发送者身份验证: 通过

```

5. 课程设计总结

本课程设计在已有 RSA / Elgamal 加密签名引擎 (基于 NTL) 的基础上, 新增实现了简单证书方案、严格层次 PKI 系统 (CA 模块、证书库模块、用户模块) 与简易安全邮件系统, 所有对象均以独立模块实现, 每个方案和协议都有独立的测试函数, 并提供了统一的演示程序 (支持文件方式输入)。

设计要点与体会：

1. 证书中被签名的内容固定为 ID || 公钥文本，签发与验证使用同一 `signed_payload()`，保证了签名的可验证性；
2. 证书公钥文本携带类型标记 (RSA/ELGAMAL)，使证书链验证时能够正确解析各级公钥，支持 RSA 与 Elgamal 混合使用；
3. 证书库在入库时即对证书做“链接到受信根”的验证，只存储 CA 发来的合法证书；查询返回完整证书路径，验证由使用方 (Bob / 邮件系统) 完成；
4. 邮件系统按任务书流程分为发送端与接收端两套流程，mlls 采用长度前缀编码，可正确传输含换行的多行邮件；任何一步 (证书链、签名) 验证失败都会拒绝处理。